

Calculador de Trastes

Documentação completa do projeto Fret Calculator API

Desenvolvedor: Santiago Ciapina Martinez Salazar

Empresa parceira: Valderrama Luthier

Projeto acadêmico desenvolvido no Instituto Mauá de Tecnologia

Repositório: <https://github.com/SantiagoCMS/fret-calculator/>

Sumário

1. Descrição Geral
2. Arquitetura do Sistema
3. Fluxo de Funcionamento
4. Funcionalidades Principais
5. Tecnologias Utilizadas
6. Infraestrutura
7. Instalação e Execução
8. Endpoints da API
9. Equipe e Autoria
10. Futuras Melhorias
11. Referências

1. Descrição Geral

O Fret Calculator API é uma API REST desenvolvida em Clojure para calcular a posição dos trastes em instrumentos de cordas com afinação temperada, como guitarras e violões. A aplicação recebe um comprimento de escala, ou uma escala pré-definida, e retorna as distâncias de cada traste em relação à pestana (nut) e à ponte (bridge).

O projeto tem como objetivo estratégico implementar um calculador de trastes no site de José Valderrama, contribuindo para atrair e engajar potenciais estudantes para o curso de formação em Lutheria que será disponibilizado. Além disso, a ferramenta busca despertar o interesse de futuros clientes em instrumentos artesanais ao demonstrar, de forma prática, os fundamentos de precisão envolvidos na construção de instrumentos.

Assim, o calculador atua como recurso funcional, material de apoio educacional e meio de divulgação do trabalho artesanal, ampliando a visibilidade do curso, da lutheria e dos produtos desenvolvidos pela empresa parceira.

2. Arquitetura do Sistema

O sistema foi organizado em uma arquitetura simples de API backend, preparada para ser consumida posteriormente por uma interface frontend. No estado atual do projeto, a camada implementada é o backend em Clojure, responsável por receber parâmetros via HTTP, validar entradas, executar os cálculos e devolver respostas em JSON.

Camada	Responsabilidade	Status
Frontend	Interface visual para entrada de escala, afinação e número de trastes. Prevista como melhoria futura.	Planejado
Backend API	Expõe endpoints REST, valida parâmetros, calcula distâncias dos trastes e retorna JSON.	Implementado
Módulo de cálculo	Contém a fórmula matemática e as funções de cálculo de distância até a pestana e até a ponte.	Implementado
Módulo de dados musicais	Mantém escalas e afinações pré-definidas para facilitar o uso da API.	Implementado

Organização dos módulos

Arquivo	Função no projeto
src/fret_calculator/core.clj	Ponto de entrada da aplicação. Inicializa o servidor HTTP na porta 3000.
src/fret_calculator/handler.clj	Define rotas, validações e handlers dos endpoints REST.
src/fret_calculator/calculator.clj	Implementa a lógica matemática de cálculo dos trastes.
src/fret_calculator/scales.clj	Armazena escalas e afinações pré-definidas.
test/fret_calculator/calculator_test.clj	Contém testes unitários da lógica de cálculo.

3. Fluxo de Funcionamento

A comunicação é feita via REST API. O usuário, ou uma futura interface frontend, envia dados como escala, comprimento customizado, afinação e quantidade de trastes. A API processa esses parâmetros, executa o cálculo e retorna um objeto JSON com os resultados.

- 1 O cliente envia uma requisição HTTP para o endpoint adequado, por exemplo GET /frets.

- 2 O backend lê os parâmetros scale, scale_length, tuning e num_frets.
- 3 A aplicação resolve o comprimento de escala: por escala pré-definida ou por valor customizado em milímetros.
- 4 O sistema valida se o comprimento é positivo e se a quantidade de trastes está entre 1 e 36.
- 5 As funções de cálculo determinam as distâncias de cada traste até a pestana e até a ponte.
- 6 A resposta é serializada em JSON e enviada ao cliente.

4. Funcionalidades Principais

- Cálculo das posições dos trastes para instrumentos de escala temperada.
- Consulta por escalas pré-definidas: Gibson, Fender / Strat, PRS e Baritone.
- Aceite de comprimento de escala customizado em milímetros.
- Definição da quantidade de trastes a calcular, com padrão de 22 trastes.
- Retorno opcional de afinações pré-definidas junto ao cálculo.
- Listagem das escalas disponíveis pelo endpoint /scales.
- Listagem das afinações disponíveis pelo endpoint /tunings.
- Respostas em JSON formatado para facilitar integração com frontend ou outros sistemas.
- Testes unitários para validar propriedades importantes da fórmula.

Fórmula utilizada

A API utiliza a fórmula padrão de lutheria para instrumentos com temperamento igual em 12 semitons. Para um traste n e uma escala de comprimento L , em milímetros:

Distância até a ponte = $L / 2^{(n/12)}$
Distância até a pestana = $L - \text{distância até a ponte}$

No 12º traste, a distância até a ponte corresponde à metade da escala, o que também é validado pelos testes unitários do projeto.

5. Tecnologias Utilizadas

Tecnologia	Versão / Biblioteca	Uso
Clojure	1.12.2	Linguagem principal da API.
Leiningen	project.clj	Gerenciamento de dependências, execução, testes e empacotamento.
Ring	ring-core, ring-jetty-adapter, ring-json	Servidor HTTP, middlewares e integração JSON.
Compojure	1.7.1	Definição das rotas REST.
Cheshire	5.12.0	Serialização JSON.
ring-mock	0.4.0	Dependência de desenvolvimento para testes.
Java	11 ou superior recomendado	Ambiente de execução da JVM.

O frontend ainda não foi implementado. A documentação considera essa camada como melhoria futura para facilitar a visualização e o uso por estudantes, clientes e equipe da lutheria.

6. Infraestrutura

Durante o desenvolvimento, a aplicação é executada localmente em `http://localhost:3000`. Para publicação, o projeto pode evoluir para um ambiente em nuvem ou servidor gerenciado, mantendo a API como serviço backend independente.

- Ambiente local: servidor Jetty iniciado pela aplicação Clojure na porta 3000.
- Deploy futuro: AWS ou Integrator, conforme decisão de infraestrutura do projeto.
- Containerização futura: Docker Compose pode ser usado para padronizar execução e facilitar deploy.
- Formato de integração: REST API com respostas JSON.

7. Instalação e Execução

Requisitos

- Java 11 ou superior.
- Leiningen instalado.
- Acesso ao repositório GitHub do projeto.

Passos de execução

```
git clone https://github.com/SantiagoCMS/fret-calculator.git
cd fret-calculator
lein deps
lein run
```

Após iniciar, o servidor fica disponível em `http://localhost:3000`.

Testes e build

```
lein test
lein uberjar
java -jar target/uberjar/fret-calculator-0.1.0-SNAPSHOT-standalone.jar
```

8. Endpoints da API

GET /frets

Calcula as distâncias dos trastes para uma escala pré-definida ou para um comprimento customizado.

Parâmetro	Tipo	Obrigatório	Descrição
scale	string	Sim*	Nome de uma escala pré-definida, como fender, gibson, prs ou baritone.
scale_length	number	Sim*	Comprimento de escala customizado em milímetros, como 660.
tuning	string	Não	Afinação pré-definida, como standard, drop-d, half-step-down, open-g, drop-c ou dadgad.
num_frets	integer	Não	Quantidade de trastes. Padrão: 22. Valor aceito: 1 a 36.

*Informe scale ou scale_length. Não é necessário informar os dois; quando scale é informado, ele tem prioridade sobre scale_length.

Exemplos de uso:

```
GET /frets?scale=fender&tuning=standard&num_frets=22
GET /frets?scale_length=660&num_frets=24
GET /frets?scale=gibson&tuning=drop-d
```

Exemplo de resposta:

```
{
  "scale-length-mm": 648.0,
  "num-frets": 3,
  "tuning": {"name": "Standard", "notes": ["E2", "A2", "D3", "G3", "B3", "E4"]},
  "frets": [
    {"fret": 1, "distance-to-nut": 36.35, "distance-to-bridge": 611.65},
    {"fret": 2, "distance-to-nut": 70.67, "distance-to-bridge": 577.33},
    {"fret": 3, "distance-to-nut": 103.05, "distance-to-bridge": 544.95}
  ]
}
```

GET /scales

Retorna as escalas pré-definidas disponíveis no sistema.

ID	Nome	Comprimento
gibson	Gibson	628.0 mm
fender	Fender / Strat	648.0 mm
prs	PRS	635.0 mm
baritone	Baritone	686.0 mm

GET /tunings

Retorna as afinações pré-definidas disponíveis no sistema.

ID	Nome	Notas
standard	Standard	E2, A2, D3, G3, B3, E4
drop-d	Drop D	D2, A2, D3, G3, B3, E4
half-step-down	Half Step Down (Eb)	Eb2, Ab2, Db3, Gb3, Bb3, Eb4
open-g	Open G	D2, G2, D3, G3, B3, D4
drop-c	Drop C	C2, G2, C3, F3, A3, D4
dadgad	DADGAD	D2, A2, D3, G3, A3, D4

Respostas de erro

Situação	Resposta
scale e scale_length ausentes ou escala inválida	{"error": "Informe scale (ex: fender) ou scale_length em mm (ex: 660)"}
Comprimento de escala não positivo	{"error": "O comprimento de escala deve ser um número positivo."}
num_frets fora do intervalo permitido	{"error": "num_frets deve estar entre 1 e 36."}
Rota inexistente	{"error": "Rota não encontrada."}

9. Equipe e Autoria

Campo	Informação
Desenvolvedor	Santiago Ciapina Martínez Salazar
Empresa parceira	Valderrama Luthier
Contexto	Projeto acadêmico do Instituto Mauá de Tecnologia e projeto de portfólio profissional.
Repositório	https://github.com/SantiagoCMS/fret-calculator/

10. Futuras Melhorias

- Implementação do frontend para melhorar a visualização das informações e permitir uso direto por estudantes, clientes e equipe da luthieria.
- Cálculo da tensão das cordas por traste, considerando o calibre da corda e demais parâmetros físicos necessários.
- Cálculo da frequência de cada nota em cada traste.
- Sugestão de afinações compatíveis com determinada escala.
- Preparação de deploy em ambiente externo, como AWS ou Integrator.
- Empacotamento com Docker Compose para padronizar instalação e execução.

11. Referências

- Repositório GitHub do projeto: <https://github.com/SantiagoCMS/fret-calculator/>
- Documentação oficial da linguagem Clojure: <https://clojure.org/>
- Leiningen: <https://leiningen.org/>
- Ring: <https://github.com/ring-clojure/ring>
- Compojure: <https://github.com/weavejester/compojure>
- Cheshire: <https://github.com/dakrone/cheshire>
- Conceito musical aplicado: temperamento igual em 12 semitons e regra de posicionamento de trastes por divisão exponencial da escala.